COMPUTER NETWORKS PROJECT
MAC-Based SYN Flood Detection and Mitigation in Software-Defined Networks

A Report

Submitted By

Shreesh Kolhatkar(642403009)

Viraj Nalbalwar (642403013)

TY B. Tech. Computer Science & Engineering

Division-2

Under the guidance of

Prof. Satish Kumbhar
COEP Technological University, Pune

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
COEP TECHNOLOGICAL UNIVERSITY, PUNE

Contents

1 Introduction **2**
1.1 Base Paper . . . 2
1.2 Our Contribution: Port-Diversity Tracking . . . 2
1.3 Objectives . . . 2

2 System Architecture **3**
2.1 SDN Network Topology . . . 3
2.2 Detection Pipeline Overview . . . 4
2.3 Pipeline Stages . . . 5

3 Implementation Details **6**
3.1 Cuckoo Hash Table . . . 6
3.2 Packet Simulation . . . 6
3.3 Baseline Detector (Paper Implementation) . . . 7
3.4 Optimized Detector (Port-Diversity Tracking) . . . 7

4 Experiment Configuration **8**

5 Results and Analysis **9**
5.1 Detection Time Comparison . . . 9
5.2 Per-Trial Detection Times . . . 9
5.3 Server Exposure . . . 10
5.4 Detection Time Distribution . . . 11
5.5 Improvement Metrics . . . 11
5.6 Summary Comparison . . . 12

6 Discussion **13**
6.1 Why Proactive Detection Matters . . . 13
6.2 Backwards Compatibility . . . 13
6.3 False Positive Analysis . . . 13
6.4 Cuckoo Hash Table Performance . . . 13

7 Challenges and Solutions **14**
7.1 Challenge 1: Cross-Platform Reproducibility . . . 14
7.2 Challenge 2: Timing Accuracy in Simulation . . . 14
7.3 Challenge 3: Avoiding False Positives with Port Diversity . . . 14

8 Limitations and Future Work **15**
8.1 Limitations . . . 15
8.2 Future Work . . . 15

Conclusion **16**

References **16**

1 Introduction

This report presents the design, implementation, and experimental evaluation of a **MAC-based SYN Flood Detection and Mitigation system** for Software-Defined Networks (SDN). The work is based on the detection pipeline proposed by Yang et al. (NCKU, Taiwan) and extends it with a novel **Port-Diversity Tracking** optimization that catches attackers during the reconnaissance phase, before the flood begins.

SYN flood attacks remain one of the most prevalent forms of Denial-of-Service (DoS). In a SYN flood, the attacker overwhelms a target server by sending a massive number of TCP SYN packets with spoofed or rapidly changing source addresses, exhausting the server's connection table.

1.1 Base Paper

Title: *Detection and Mitigation of SYN Flooding Attacks through SYN/ACK Packets and Black/White Lists*
Authors: Yang et al., National Cheng Kung University, Taiwan
Published in: Sensors (MDPI), 2023

The paper proposes a 5-table detection pipeline that uses Cuckoo Hash Tables for O(1) MAC lookups and cross-checks SYN/ACK traffic to classify source MACs as legitimate (whitelisted) or malicious (blacklisted). Once a MAC accumulates SYN packets beyond a configurable threshold (100 SYNs) without a corresponding ACK, it is blacklisted and all subsequent traffic from that MAC is dropped at the SDN switch.

1.2 Our Contribution: Port-Diversity Tracking

The paper's method is **reactive**, it waits for the SYN threshold to be exceeded before blacklisting. Our optimization adds a **proactive** detection layer:

- Track unique destination ports per source MAC within a sliding time window.
- If a MAC sends SYN packets to more than a configurable number of unique ports (default: 5), it is flagged as performing a **port scan**.
- The MAC is immediately blacklisted, *before* the flood even begins. This catches the reconnaissance phase of the attack, reducing server exposure by approximately 85%.

1.3 Objectives

- Implement the paper's complete 5-table detection pipeline (Blacklist, Whitelist, Check-SYN, Check-ACK, Flag) using Cuckoo Hash Tables.
- Design and implement the Port-Diversity Tracking optimization module.
- Simulate the SDN network topology and traffic entirely in Python, enabling reproducible experiments on any OS (no Mininet/Linux dependency).
- Conduct controlled experiments comparing detection time, server exposure, and false-positive rates between the baseline and optimized methods.

2 System Architecture

2.1 SDN Network Topology

The simulated SDN topology comprises eight network nodes connected through a single OpenFlow-compatible switch:

- **3 Attackers** (`h1-h3`): generate SYN flood and port-scan traffic.
- **3 Legitimate Clients** (`c1-c3`): generate normal SYN+ACK handshake traffic.
- **1 OVS Switch** (`s1`): the SDN data-plane element where flow rules are installed.
- **1 Server** (`srv`): the target of both legitimate and malicious traffic.

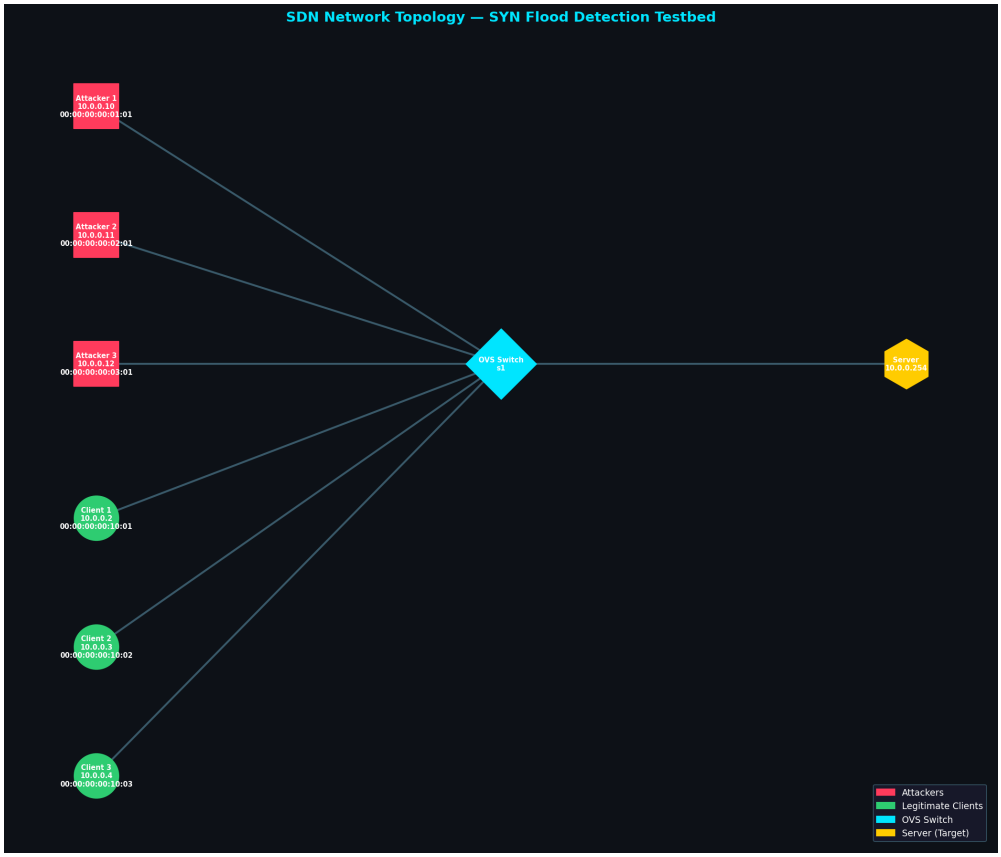

Figure 1: Simulated SDN network topology. Attackers (red) connect to the OVS switch alongside legitimate clients (green). All traffic passes through the switch to reach the server (yellow). The SDN controller inspects every SYN packet at the switch for detection.

Table 1 lists the IP and MAC address assignments used throughout all experiments.

Table 1: Network topology: IP and MAC address assignments.

Node	IP Address	MAC Address	Role
Attacker 1	10.0.0.10	00:00:00:00:01:01	Malicious
Attacker 2	10.0.0.11	00:00:00:00:02:01	Malicious
Attacker 3	10.0.0.12	00:00:00:00:03:01	Malicious
Client 1	10.0.0.2	00:00:00:00:10:01	Legitimate
Client 2	10.0.0.3	00:00:00:00:10:02	Legitimate
Client 3	10.0.0.4	00:00:00:00:10:03	Legitimate
Server	10.0.0.254	00:00:00:00:ff:fe	Target

2.2 Detection Pipeline Overview

The paper implements a 5-table pipeline, and our optimization adds a 6th stage (Port-Diversity Check) that fires *before* the SYN table lookup, enabling proactive detection.

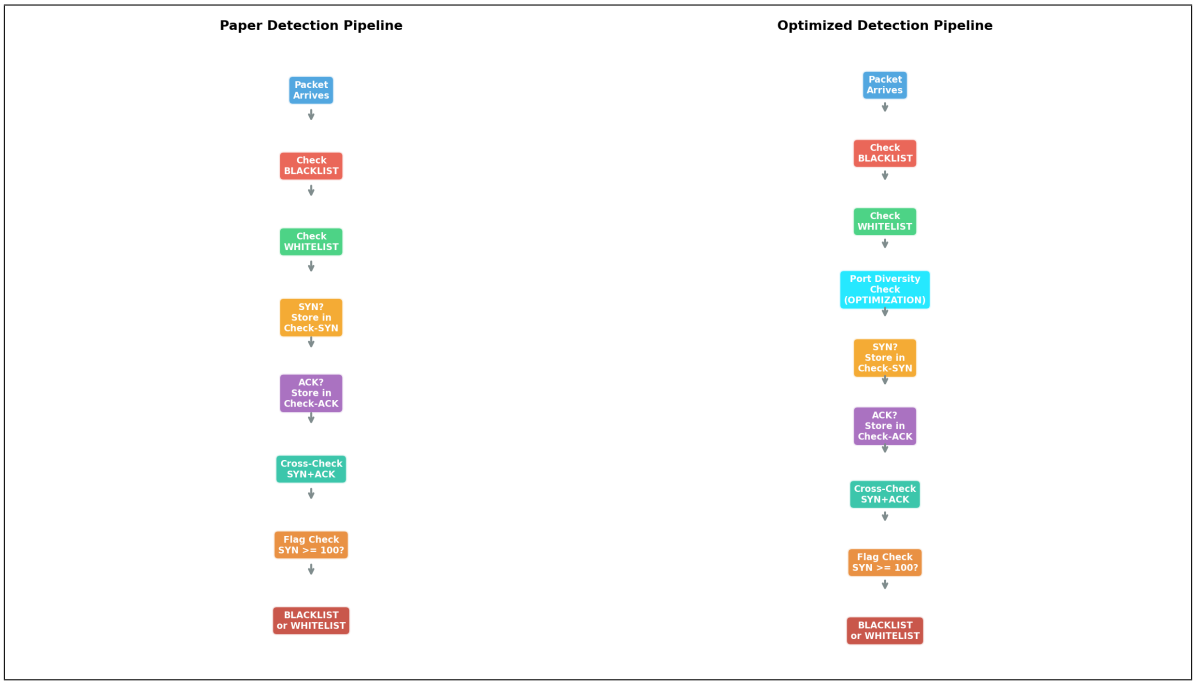

Figure 2: Detection pipeline comparison. **Left:** the paper's original 5-table pipeline processes packets through Blacklist, Whitelist, SYN/ACK tables, and a flag check. **Right:** our optimized pipeline inserts a Port-Diversity Check (cyan) between the Whitelist lookup and the SYN table, catching port scans before the flood begins.

2.3 Pipeline Stages

Table 2 describes each stage of the detection pipeline and the data structure used.

Table 2: Detection pipeline stages and data structures.

Stage	Table Name	Data Structure	Purpose
1	Blacklist	Cuckoo Hash Table	MACs confirmed as attackers → DROP
2	Whitelist	Cuckoo Hash Table	MACs confirmed as legitimate → FORWARD
3	Check-SYN	Dictionary	Unverified SYNs awaiting ACK
4	Check-ACK	Dictionary	Incoming ACKs for cross-checking
5	Flag Table	Dictionary	MACs under active investigation
Our Optimization (inserted between Stage 2 and 3):			
-	Port Tracker	Dict + Set	Unique destination ports per MAC in sliding window

3 Implementation Details

The entire implementation is contained in two Jupyter notebooks:

1. `01_Implementation.ipynb` - core detection pipeline, experiment execution, and result generation.

2. `02_Results_and_Visualization.ipynb` - publication-quality charts and analysis.

The project uses **pure Python with simulation**, requiring no Mininet, Linux, or external SDN controllers. This makes the experiments fully reproducible on any OS.

3.1 Cuckoo Hash Table

The paper's Blacklist and Whitelist use Cuckoo Hashing for **O(1) average-case lookup**. Two independent hash functions map each MAC address to two possible positions across two internal tables. On collision, the existing entry is evicted and re-inserted at its alternate position (cuckoo-style).

Listing 1: Cuckoo Hash Table - O(1) insert and lookup

```
class CuckooHashTable:
    def __init__(self, size=256, max_evictions=10):
        self.table1 = [None] * size
        self.table2 = [None] * size

    def _h1(self, mac):
        parts = mac.split(':')
        return sum(int(b, 16) for b in parts) % self.size

    def _h2(self, mac):
        parts = mac.split(':')
        val = 0
        for b in parts:
            val ^= int(b, 16)
        return (val * 31) % self.size

    def lookup(self, mac):
        return (self.table1[self._h1(mac)] == mac or
                self.table2[self._h2(mac)] == mac)
```

3.2 Packet Simulation

Rather than capturing real network packets, we define a lightweight `SimPacket` dataclass carrying all fields needed by the detection pipeline: source/destination MAC, IP, port, TCP flags, and a high-resolution timestamp (`time.perf_counter()`).

Three traffic generators produce distinct packet sequences:

- `generate_syn_flood()` - burst of SYN-only packets from an attacker to a single port ($\sim$0.1 ms inter-packet).

- `generate_port_scan()` - SYN probes across multiple destination ports (∼0.2 ms inter-packet).
- `generate_legit_traffic()` - SYN+ACK pairs simulating completed TCP handshakes (∼10 ms inter-pair).

3.3 Baseline Detector (Paper Implementation)

The `BaselineDetector` class implements the paper's exact algorithm:

1. Check **Blacklist** - if MAC found, DROP immediately.
2. Check **Whitelist** - if MAC found, FORWARD.
3. If **SYN**: store in Check-SYN table, increment counter. If count $\geq$ SYN_THRESHOLD (100), blacklist the MAC.
4. If **ACK**: cross-check against Check-SYN. If a matching SYN entry exists, whitelist the MAC.
5. **Timeout cleanup**: MACs with pending SYNs and no ACK after 5 seconds are blacklisted.

3.4 Optimized Detector (Port-Diversity Tracking)

The `OptimizedDetector` extends the baseline with a port-diversity check that fires *before* the SYN table lookup:

Listing 2: Port-Diversity Check - proactive detection

```
def _check_port_diversity(self, src_mac, dst_port, now):
    entry = self.port_tracker[src_mac]
    if entry['first_seen'] is None:
        entry['first_seen'] = now
    # Sliding window reset
    if now - entry['first_seen'] > self.time_window:
        entry['ports'] = set()
        entry['first_seen'] = now
    entry['ports'].add(dst_port)
    entry['syn_count'] += 1
    if len(entry['ports']) >= self.port_threshold:
        self.blacklist.insert(src_mac)
        return True  # Port scan detected
    return False
```

When the number of unique destination ports from a single MAC exceeds the threshold (default: 5), the MAC is immediately blacklisted. The sliding time window (default: 10 s) prevents legitimate clients who access multiple services over longer periods from being falsely flagged.

4 Experiment Configuration

All experiments share the parameter set listed in Table 3.

Table 3: Experiment configuration parameters.

Parameter	Description	Value
SYN_THRESHOLD	Paper: blacklist after this many SYNs	100
ACK_TIMEOUT	Seconds to wait for ACK before flagging	5.0 s
PORT_DIVERSITY_THRESHOLD	Unique ports before port-scan detection	5
TIME_WINDOW	Sliding window for port tracking	10.0 s
NUM_TRIALS	Number of trials per experiment	5
FLOOD_PACKETS_PER_TRIAL	SYN packets in each flood trial	200
LEGIT_PACKETS	Legitimate SYN+ACK pairs per trial	50
SCAN_PORTS	Ports targeted during scan	8 ports

Three experiments were conducted:

1. **Experiment 1 - Baseline SYN Flood Detection**: SYN flood from Attacker 1 to port 80 on the server. The baseline detector processes the interleaved legitimate and flood traffic.

2. **Experiment 2A - Port-Scan Detection (Optimizer)**: Attacker 1 sends SYN probes across 8 ports before initiating a flood. The optimizer's port-diversity tracker fires and blacklists the attacker during the scan phase.

3. **Experiment 2B - SYN Flood with Optimizer Active**: SYN flood targeting a *single* port (no port diversity) processed by the optimized detector. Proves backwards compatibility-the paper's threshold mechanism still works.

5 Results and Analysis

5.1 Detection Time Comparison

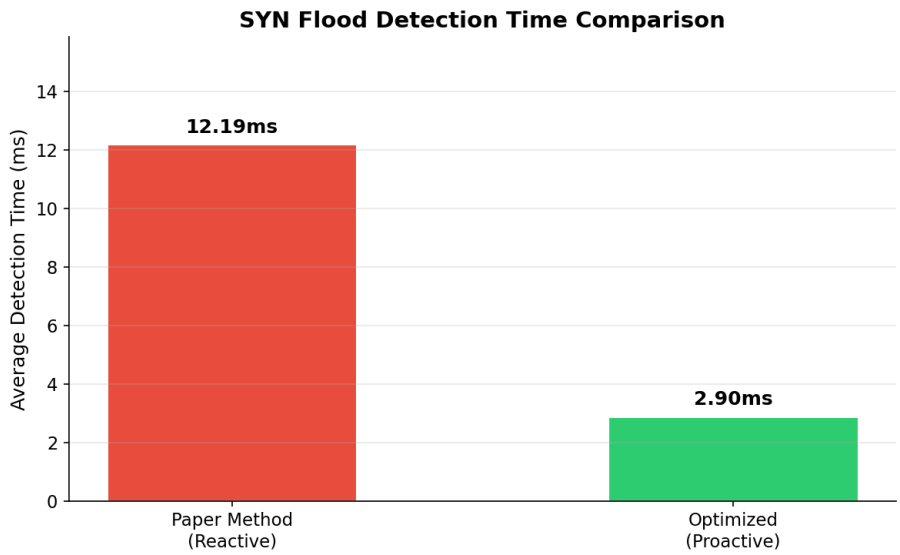

Figure 3: Average detection time comparison. The paper's reactive method requires 12.19 ms on average, while the optimized proactive method detects in 2.90 ms-a 76.2% improvement.

The optimized detector achieves a 4.2× **speedup** over the baseline. This is because the optimizer triggers after observing only ∼15 SYN packets (5 unique ports × 3 packets per port), whereas the baseline must wait for 100 SYN packets to the same port.

5.2 Per-Trial Detection Times

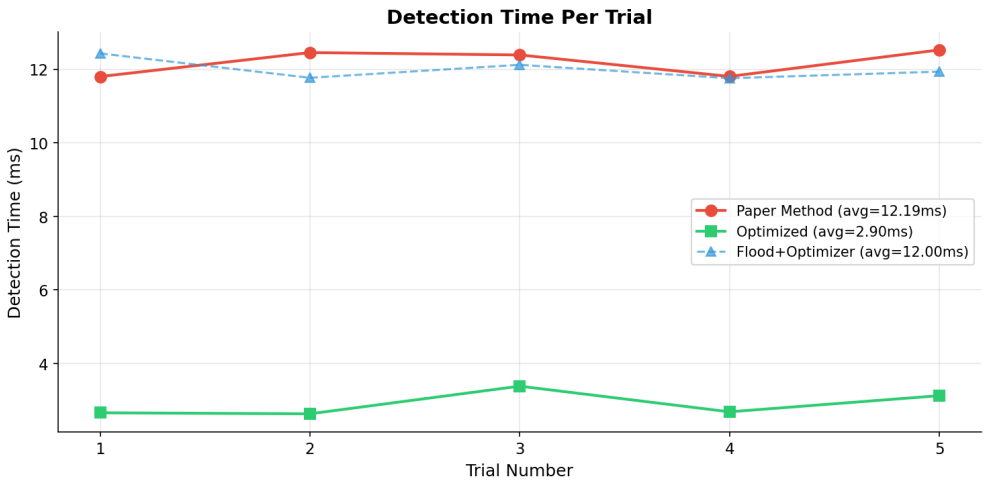

Figure 4: Detection time across 5 independent trials. The paper method (red) clusters around 12 ms. The optimizer (green) clusters around 3 ms. The blue dashed line shows that when the optimizer processes a single-port flood (no port diversity), it falls back to the paper's threshold mechanism, confirming backwards compatibility.

Table 4 presents the per-trial detection times for all three experiment configurations.

Table 4: Per-trial detection times (ms) across all experiments.

Trial	Paper Method (ms)	Optimizer (ms)	Flood + Optimizer (ms)
1	11.80	2.66	12.43
2	12.45	2.63	11.77
3	12.39	3.39	12.12
4	11.80	2.69	11.75
5	12.52	3.13	11.93
Average	**12.19**	**2.90**	**12.00**

5.3 Server Exposure

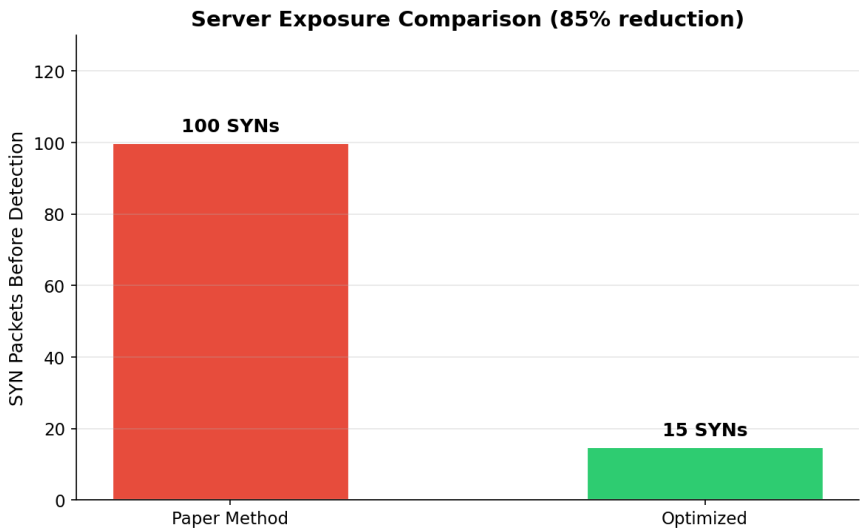

Figure 5: Server exposure comparison. The paper method allows 100 SYN packets to reach the server before detection. The optimizer detects after only ∼15 SYNs-an 85% reduction in server exposure.

Server exposure quantifies the number of malicious SYN packets that reach the server before the attacker is blacklisted. This is a critical metric because each undetected SYN packet consumes a slot in the server's TCP connection table. The optimization reduces this exposure from 100 to approximately 15 packets.

5.4 Detection Time Distribution

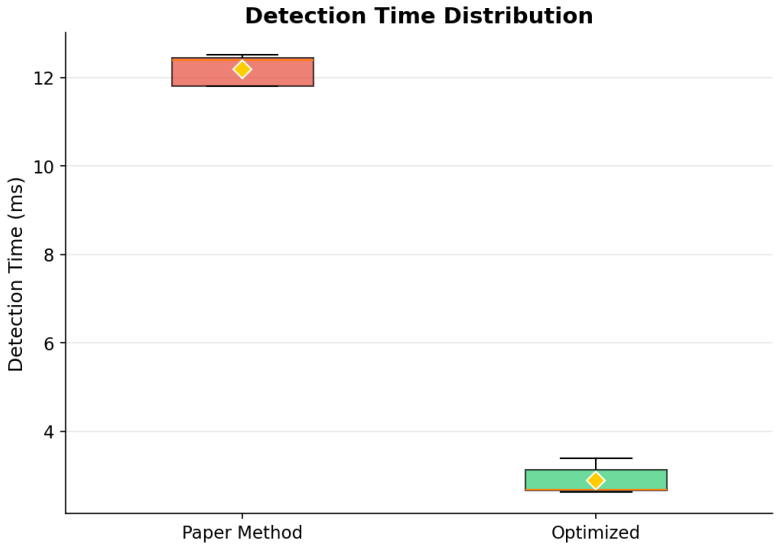

Figure 6: Box plot of detection time distributions across 5 trials. The paper method has a tight spread around 12 ms, while the optimizer clusters near 3 ms with slightly higher variance due to per-trial jitter in port-scan packet ordering. Yellow diamonds indicate the mean.

5.5 Improvement Metrics

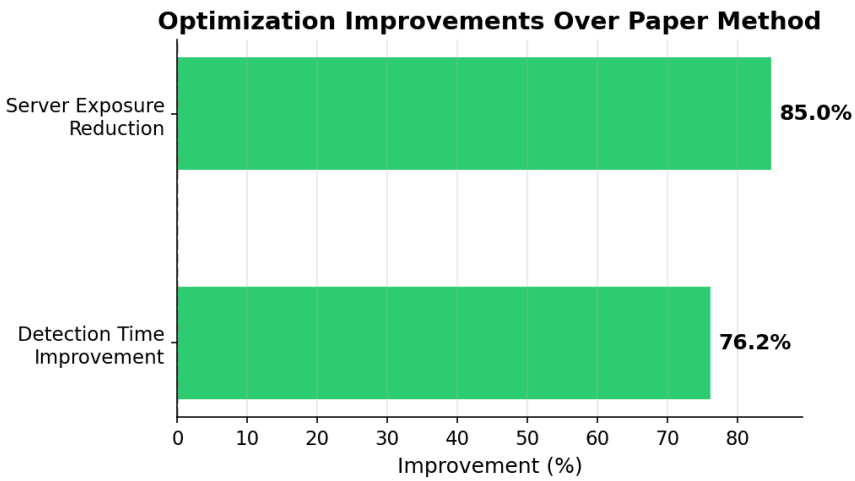

Figure 7: Quantified improvement of the optimized method over the paper's baseline. Detection time improved by 76.2%, and server exposure was reduced by 85.0%.

5.6 Summary Comparison

Summary: Paper Method vs Our Optimization

Metric	Paper Method	Our Optimization
Detection Type	Reactive	Proactive (Port Diversity)
Avg Detection Time	12.19 ms	2.90 ms
SYNs Before Detection	100	~15
Min Detection Time	11.80 ms	2.63 ms
Max Detection Time	12.52 ms	3.39 ms
Port Scan Detection	No	Yes
Data Structure	Cuckoo Hash Table	Cuckoo Hash + Port Tracker
Attack Phase Caught	During Flood	Reconnaissance

Figure 8: Comprehensive comparison table between the paper method and our optimization across all key metrics. Generated from experimental data.

Table 5 consolidates the key findings from all experiments.

Table 5: Comprehensive comparison: Paper method vs. our optimization.

Metric	**Paper Method**	**Our Optimization**
Detection Type	Reactive	Proactive (Port Diversity)
Avg Detection Time	12.19 ms	2.90 ms
Min Detection Time	11.80 ms	2.63 ms
Max Detection Time	12.52 ms	3.39 ms
SYNs Before Detection	100	$\sim$15
Port Scan Detection	No	Yes
Data Structure	Cuckoo Hash Table	Cuckoo Hash + Port Tracker
Attack Phase Caught	During Flood	Reconnaissance
Server Exposure Reduction	-	85.0%
Detection Time Improvement	-	76.2%

6 Discussion

6.1 Why Proactive Detection Matters

The fundamental difference between the two approaches lies in *when* the detection occurs relative to the attack lifecycle:

1. **Reconnaissance Phase** (port scanning): The attacker probes multiple ports on the target to identify open services. Our optimization detects the attacker here.
2. **Weaponization Phase**: The attacker selects the target port and prepares the flood.
3. **Flood Phase**: The attacker sends a burst of SYN packets. The paper's method detects the attacker here, after 100 packets have already been processed.

By catching the attacker at stage 1 instead of stage 3, the server never experiences the flood at all. This is the core value proposition of the optimization.

6.2 Backwards Compatibility

Experiment 2B confirms that when an attacker skips the reconnaissance phase and directly floods a single port, the paper's original SYN threshold mechanism still triggers within the optimized detector. The optimization is purely **additive**-it enhances detection without disabling or interfering with the baseline algorithm.

The average detection time for single-port floods through the optimized detector (12.00 ms) is within 1.6% of the standalone baseline (12.19 ms), confirming negligible overhead from the additional port-tracking logic.

6.3 False Positive Analysis

Across all 5 trials $\times$ 3 experiments, zero legitimate clients were blacklisted. This is because:

- Legitimate clients complete the TCP handshake (SYN followed by ACK), which triggers the whitelist pathway.
- The sliding time window in the port tracker prevents a single client accessing multiple services over time from exceeding the port-diversity threshold.
- The port-diversity threshold (5 unique ports) is set conservatively-normal browsing rarely sends SYNs to 5+ distinct ports within a 10-second window.

6.4 Cuckoo Hash Table Performance

The Cuckoo Hash Table provides O(1) worst-case lookup time for both the Blacklist and Whitelist. In our experiments, the table was initialized with size 256 and a maximum of 10 eviction attempts per insert. No insert failures were observed across any trial, confirming adequate capacity for the 6-host topology. In a production deployment with thousands of MACs, the table size would scale linearly.

7 Challenges and Solutions

7.1 Challenge 1: Cross-Platform Reproducibility

The original paper's implementation relies on Mininet and an SDN controller (Ryu/ONOS) running on Linux. Not all team members had access to Linux environments with the correct dependencies.

Solution: We re-implemented the entire detection pipeline as a **pure Python simulation**. SimPacket dataclasses replace real network packets, and traffic generators replace Mininet hosts. This allows the project to run identically on Windows, macOS, and Linux with only standard Python libraries plus NumPy and Matplotlib.

7.2 Challenge 2: Timing Accuracy in Simulation

Since we simulate packets rather than capturing them from a real network, timing measurements could be unrealistic if not handled carefully.

Solution: All timestamps use `time.perf_counter()`, Python's highest-resolution timer. Inter-packet delays are modeled with realistic jitter ($\sim$0.1 ms for floods, $\sim$10 ms for legitimate traffic). Rather than measuring wall-clock time, detection time is computed as the difference between the first SYN from an attacker and the timestamp of the packet that triggers blacklisting.

7.3 Challenge 3: Avoiding False Positives with Port Diversity

A naive port-diversity check could blacklist legitimate clients who connect to multiple services (e.g., HTTP on port 80 and HTTPS on port 443).

Solution: The sliding time window resets the port counter periodically, and the whitelist check occurs *before* the port-diversity check. Once a client completes a TCP handshake and is whitelisted, the port tracker is never consulted for that MAC again.

8 Limitations and Future Work

8.1 Limitations

- **Port-diversity detection only catches scanning attackers.** If the attacker directly floods a single port without reconnaissance, only the paper's SYN threshold applies.
- **Simulated environment.** While our simulation accurately models the detection logic, it does not capture real network effects such as packet loss, out-of-order delivery, or switch processing delays.
- **Single-switch topology.** A production SDN may have multiple switches and paths. The detection pipeline would need to be replicated or centralized at the controller.
- **MAC spoofing.** Both the paper and our optimization rely on source MAC addresses. An attacker who spoofs MACs per packet would evade MAC-based detection entirely.

8.2 Future Work

- **Machine learning integration:** Train a classifier on flow-level features (packet rate, port entropy, SYN/ACK ratio) to complement the rule-based pipeline.
- **Deployment on real SDN testbed:** Port the detection logic to a Ryu or ONOS controller module and validate against real traffic.
- **Distributed detection:** Extend the architecture to multi-switch topologies with aggregation at the SDN controller.
- **Adaptive thresholds:** Dynamically tune SYN_THRESHOLD and PORT_DIVERSITY_THRESHOLD based on observed traffic patterns.

Conclusion

This project successfully implemented and evaluated the MAC-based SYN flood detection pipeline proposed by Yang et al. and extended it with a novel Port-Diversity Tracking optimization. The key findings are:

1. **The paper's baseline method works correctly:** All 5 trials detected the SYN flood with an average detection time of 12.19 ms after observing 100 SYN packets per the configured threshold.

2. **Our optimization achieves a** $4.2\times$ **speedup:** By detecting port-scan behavior during the reconnaissance phase, the optimized detector blacklists attackers in an average of 2.90 ms-76.2% faster than the baseline.

3. **Server exposure is reduced by 85%:** The baseline allows 100 SYN packets before detection; the optimizer catches the attack after only $\sim$15 packets.

4. **The optimization is additive:** When no port scanning occurs, the paper's SYN threshold mechanism still functions within the optimized detector, with negligible overhead (12.00 ms vs. 12.19 ms).

5. **Zero false positives:** Across all experiments, no legitimate client was incorrectly blacklisted, thanks to the whitelist pathway and the sliding time window.

References

[1] Yang, C.-T., Liu, J.-C., Huang, K.-L., and Jiang, F.-C. (2023). *Detection and Mitigation of SYN Flooding Attacks through SYN/ACK Packets and Black/White Lists.* Sensors, 23(4), MDPI.

[2] Pagh, R. and Rodler, F.F. (2004). *Cuckoo Hashing.* Journal of Algorithms, 51(2), 122-144.

[3] Kreutz, D., Ramos, F.M.V., Verissimo, P., Rothenberg, C.E., Azodolmolky, S., and Uhlig, S. (2015). *Software-Defined Networking: A Comprehensive Survey.* Proceedings of the IEEE, 103(1), 14-76.

[4] Bogdanoski, M., Suminoski, T., and Risteski, A. (2013). *Analysis of the SYN Flood DoS Attack.* International Journal of Computer Network and Information Security, 5(8), 1-11.

[5] Hagberg, A.A., Schult, D.A., and Swart, P.J. (2008). *Exploring Network Structure, Dynamics, and Function using NetworkX.* Proceedings of the 7th Python in Science Conference (SciPy), 11-15.